الفهرس

React.js — The Complete Easy Guide

الرقم	عنوان القسم	الصفحة
0.01	**How to Use This Guide**	1
0.02	**Table of Contents**	1
0.03	**1. What is React?**	2
0.03.01	Why people like React	2
0.03.02	Library vs Framework	2
0.04	**2. Setup Your First Project**	3
0.04.01	Create a project with Vite (fast and modern)	3
0.04.02	What each command does	3
0.05	**3. Project Structure**	3
0.05.01	`main.jsx` — the start point	4
0.06	**4. JSX**	4
0.06.01	Rule 1: Return ONE parent element	4
0.06.02	Rule 2: Use `{}` for JavaScript	5
0.06.03	Rule 3: Close every tag	5
0.06.04	Rule 4: Some names are different	5
0.06.05	Rule 5: Style is an object	5
0.06.06	Rule 6: Comments	5
0.07	**5. Components**	6
0.07.01	Example 1: Simple component	6
0.07.02	Example 2: Use a component	6
0.07.03	Example 3: Arrow function style	6
0.07.04	Example 4: Components inside components	6
0.07.05	Export types	7
0.08	**6. Props**	7
0.08.01	Example 1: One prop	7
0.08.02	Example 2: Destructuring (cleaner)	7
0.08.03	Example 3: Many props	8
0.08.04	Strings vs other types	8
0.08.05	Example 4: Default values	8
0.08.06	Example 5: Send a function	8

	as a prop	
0.08.07	Example 6: The spread operator	8
0.08.08	🟢 Golden Rule: Props are READ-ONLY	9
0.09	**7. 🧒 Children**	9
0.09.01	Example 1: A Box component	9
0.09.02	Example 2: A Card with a title	9
0.09.03	Example 3: A layout component	10
0.10	**8. 🧠 State with useState**	10
0.10.01	Basic syntax	10
0.10.02	Example 1: Counter	10
0.10.03	Example 2: Text state	11
0.10.04	Example 3: Boolean state (show / hide)	11
0.10.05	Example 4: Object state	11
0.10.06	Example 5: Array state	11
0.10.07	Example 6: Update using the previous value	12
0.10.08	🟢 Never change state directly	12
0.11	**9. 👆 Events**	13
0.11.01	Common events	13
0.11.02	Example 1: Click	13
0.11.03	Example 2: Inline arrow function	13
0.11.04	Example 3: Send an argument	13
0.11.05	Example 4: The event object	13
0.11.06	Example 5: Key press	14
0.11.07	Example 6: Stop the page refresh	14
0.11.08	⚠️ Big mistake: calling the function	14
0.12	**10. 🔀 Conditional Rendering**	14
0.12.01	Way 1: `if` statement	15
0.12.02	Way 2: Ternary `? :`	15
0.12.03	Way 3: `&&` (show or nothing)	15
0.12.04	Way 4: Variable	15
0.12.05	Example: Loading / Error /	15

	Data	
0.12.06	Example: Login button	15
0.13	**11. 📋 Lists and Keys**	16
0.13.01	Example 1: Simple list	16
0.13.02	Example 2: List of objects	16
0.13.03	Example 3: Use your own component	16
0.13.04	Example 4: Filter then map	17
0.13.05	Example 5: Empty list message	17
0.13.06	🔑 What is a key?	17
0.14	**12. 📝 Forms**	17
0.14.01	Example 1: One input	17
0.14.02	Example 2: Many inputs, one state object	18
0.14.03	Example 3: Checkbox	18
0.14.04	Example 4: Select (dropdown)	19
0.14.05	Example 5: Radio buttons	19
0.14.06	Example 6: Simple validation	19
0.15	**13. ⏱ useEffect**	20
0.15.01	Syntax	20
0.15.02	The dependency array	20
0.15.03	Example 1: Run once	20
0.15.04	Example 2: Change the page title	21
0.15.05	Example 3: Timer with cleanup	21
0.15.06	Example 4: Listen to the window	21
0.15.07	Example 5: Save to localStorage	21
0.15.08	⚠ Infinite loop	21
0.15.09	💡 You may not need an effect	22
0.16	**14. 🌐 Fetching Data**	22
0.16.01	Example 1: Basic fetch	22
0.16.02	The 3 states of every request	23
0.16.03	Example 2: Fetch when an ID changes	23
0.16.04	Example 3: Send data with POST	23
0.17	**15. 📌 useRef**	24

0.17.01	Use 1: Focus an input	**24**
0.17.02	Use 2: Keep a value between renders	**24**
0.17.03	Use 3: Save a timer ID	**24**
0.17.04	`useState` vs `useRef`	**25**
0.18	**16. 🌍 useContext**	**25**
0.18.01	Step 1: Create the context	**25**
0.18.02	Step 2: Wrap your app	**26**
0.18.03	Step 3: Use it anywhere	**26**
0.18.04	Example: User context	**26**
0.19	**17. 🔄 useReducer**	**27**
0.19.01	Syntax	**27**
0.19.02	Example 1: Counter	**27**
0.19.03	Example 2: Shopping cart	**27**
0.19.04	When to use which?	**28**
0.20	**18. 🏎️ Performance: memo, useMemo, useCallback**	**28**
0.20.01	`React.memo` — skip re-drawing if props are the same	**28**
0.20.02	`useMemo` — remember a calculated value	**29**
0.20.03	`useCallback` — remember a function	**29**
0.20.04	Quick summary	**29**
0.20.05	Other easy performance tips	**30**
0.21	**19. □ Custom Hooks**	**30**
0.21.01	Example 1: `useToggle`	**30**
0.21.02	Example 2: `useFetch`	**31**
0.21.03	Example 3: `useLocalStorage`	**31**
0.21.04	Example 4: `useDebounce` (good for search boxes)	**32**
0.22	**20. 🛞 React Router**	**32**
0.22.01	Install	**32**
0.22.02	Example 1: Basic routes	**32**
0.22.03	Example 2: URL parameters	**33**
0.22.04	Example 3: Go to another page with code	**33**
0.22.05	Example 4: Active link style	**33**
0.22.06	Example 5: Protected page	**34**
0.22.07	Example 6: Nested routes	**34**
0.23	**21. 🎨 Styling**	**34**
0.23.01	Way 1: Normal CSS file	**34**

0.23.02	Way 2: Inline style	**35**
0.23.03	Way 3: CSS Modules (no name conflicts)	**35**
0.23.04	Way 4: Tailwind CSS (utility classes)	**35**
0.23.05	Dynamic classes	**35**
0.23.06	Which one should I choose?	**36**
0.24	**22. 🔝 Lifting State Up**	**36**
0.24.01	Example: Two components share a temperature	**36**
0.24.02	Example: Search + List	**37**
0.25	**23. 📜 Rules of Hooks**	**37**
0.25.01	Rule 1: Call hooks only at the **top level**	**37**
0.25.02	Rule 2: Call hooks only in **React functions**	**38**
0.26	**24. ⚠️ Common Mistakes**	**38**
0.26.01	Mistake 16: State does not change immediately	**38**
0.27	**25. 🏆 Mini Project: Todo App**	**39**
0.27.01	`App.jsx`	**39**
0.27.02	`App.css` (dark and gold ❤️🤍)	**41**
0.27.03	What did we use?	**42**
0.28	**26. 📄 Cheat Sheet**	**42**
0.28.01	Hooks	**42**
0.28.02	Patterns	**42**
0.28.03	File flow	**43**
0.29	**27. 📖 Glossary: English ↔ Egyptian Arabic**	**43**
0.30	**28. 🏏 Practice Exercises**	**44**
0.30.01	🟢 Easy	**44**
0.30.02	🟡 Medium	**44**
0.30.03	🔴 Hard	**44**
0.31	**29. 📬 Next Steps**	**44**
0.31.01	Good habits ☑️	**45**
0.31.02	⚛️ Happy coding! ❤️🤍	**45**

Level: A2 English (simple words, short sentences) **Extra help:** EG Egyptian Arabic notes, always with English next to them **Goal:** Learn React from zero, with many small examples for every point.

0.01 How to Use This Guide

Symbol	Meaning
🔘	Important idea
💻	Code example
EG	Egyptian Arabic explanation
⚠️	Common mistake
✅	Good way
❌	Bad way
🛠️	Try it yourself

Table 1

Tip: Read one section, then type the examples yourself. Do not only copy and paste.

0.02 Table of Contents

1. [What is React?](#1--what-is-react)
2. [Setup Your First Project](#2--setup-your-first-project)
3. [Project Structure](#3--project-structure)
4. [JSX](#4--jsx)
5. [Components](#5--components)
6. [Props](#6--props)
7. [Children](#7--children)
8. [State with useState](#8--state-with-usestate)
9. [Events](#9--events)
10. [Conditional Rendering](#10--conditional-rendering)
11. [Lists and Keys](#11--lists-and-keys)
12. [Forms](#12--forms)
13. [useEffect](#13--useeffect)
14. [Fetching Data](#14--fetching-data)
15. [useRef](#15--useref)
16. [useContext](#16--usecontext)
17. [useReducer](#17--usereducer)
18. [Performance: memo, useMemo, useCallback](#18--performance-memo-usememo-usecallback)
19. [Custom Hooks](#19--custom-hooks)

20. [React Router](#20--react-router)
21. [Styling](#21--styling)
22. [Lifting State Up](#22--lifting-state-up)
23. [Rules of Hooks](#23--rules-of-hooks)
24. [Common Mistakes](#24--common-mistakes)
25. [Mini Project: Todo App](#25--mini-project-todo-app)
26. [Cheat Sheet](#26--cheat-sheet)
27. [Glossary: English ↔ Egyptian Arabic](#27--glossary-english--egyptian-arabic)
28. [Practice Exercises](#28--practice-exercises)
29. [Next Steps](#29--next-steps)

0.03 1. ⚛ What is React?

React is a JavaScript library. We use it to build user interfaces (UI).

A user interface is what the user sees and clicks on a website or app.

🔘 **Main idea: You build small pieces. Then you put the pieces together.**

```
      App
     /   \
Header    Main
            |
        ProductList
         /   |   \
      Card Card Card
```

0.03.01 Why people like React

- ✅ ****Components:** small pieces you can use again and again.**
- ✅ ****Fast updates:** React changes only the part of the screen that changed.**
- ✅ ****Big community:** many tutorials, tools, and jobs.**
- ✅ ****Works for web and mobile** (React Native).**

(user interfaces). بنبني بيها واجهات المستخدم **(JavaScript library)** هي مكتبة جافاسكريبت **React **:****بالمصري EG
)كومبوننتس(، زي ****Components**** بدل ما نكتب صفحة كبيرة واحدة، بنقسمها لحتت صغيرة اسمها
(LEGO). *In English: React is a JavaScript library. We build the UI from small pieces قطع الليجو
called components, like LEGO.*

0.03.02 Library vs Framework

	Library (React)	Framework (Angular)
Control	You choose the tools	It gives you most tools
Size	Small	Bigger
Learning	Easier to start	More rules

Table 2

0.04 2. 🚀 Setup Your First Project

You need Node.js first. Download it from the Node.js website.

Check that it works:

```
node -v
npm -v
```

0.04.01 Create a project with Vite (fast and modern)

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Open the link in your browser. Usually it is `http://localhost:5173`.

0.04.02 What each command does

Command	Meaning
`npm create vite@latest`	Make a new project
`cd my-app`	Go inside the folder
`npm install`	Download the packages
`npm run dev`	Start the development server
`npm run build`	Make the final files for production

Table 3

> على جهازك .كل ما تعدّل في **(local server)** بيشغّل السيرفر المحلي **`npm run dev`** بالمصري **:الأمر** EG
> الكود، الصفحة بتتحدّث لوحدها **(hot reload). *In English: `npm run dev` starts a local server. When you change the code, the page updates by itself.***

🖌 Try it: Open `src/App.jsx`**. Change the text. Save. Look at the browser.**

0.05 3. 🗂 Project Structure

```
my-app/
├── public/            ← Files like images and icons
├── src/
│   ├── assets/        ← Your images
│   ├── components/    ← Your components (you make this)
│   ├── App.jsx        ← Main component
│   ├── main.jsx       ← Start point
│   └── index.css      ← Styles
├── index.html         ← One HTML page
├── package.json       ← List of packages and scripts
└── vite.config.js     ← Vite settings
```

0.05.01 `main.jsx` — the start point

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Line by line:

30. **We import React tools.**
31. **We find the `<div id="root">` in `index.html`.**
32. **We put our `<App />` inside it.**

> بالمصري** EG **:** `main.jsx` بياخد الكومبوننت .هو أول ملف بيشتغل `App` ويحطه جوه الـ `div` اللي بتاعه **id** `root`. *In English: `main.jsx` runs first. It puts `App` inside the div with id `root`.*

0.06 4. ⚛ JSX

JSX looks like HTML, but it lives inside JavaScript.

```
const element = <h1>Hello, React!</h1>;
```

0.06.01 Rule 1: Return ONE parent element

✕ Bad

```
function Bad() {
  return (
    <h1>Title</h1>
    <p>Text</p>
  );
}
```

✅ Good — use a `div`

```
function Good() {
  return (
    <div>
      <h1>Title</h1>
      <p>Text</p>
    </div>
  );
}
```

✅ Better — use a Fragment (no extra div)

```
function Better() {
  return (
```

```
    <>
      <h1>Title</h1>
      <p>Text</p>
    </>
  );
}
```

0.06.02 Rule 2: Use `{ }` for JavaScript

```
function Welcome() {
  const name = "Ahmed";
  const age = 25;

  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>Next year you will be {age + 1}.</p>
      <p>Name in capital letters: {name.toUpperCase()}</p>
    </div>
  );
}
```

0.06.03 Rule 3: Close every tag

```
<img src="car.png" alt="A red car" />
<input type="text" />
<br />
```

0.06.04 Rule 4: Some names are different

HTML	JSX
class	className
for	htmlFor
onclick	onClick
tabindex	tabIndex

Table 4

```
<label htmlFor="email" className="label">Email</label>
<input id="email" type="email" />
```

0.06.05 Rule 5: Style is an object

```
<p style={{ color: "gold", fontSize: "20px" }}>Gold text</p>
```

Two curly braces: the **outside `{ }` means "JavaScript here". The **inside** `{ }` is the style object.**

0.06.06 Rule 6: Comments

```
<div>
  {/* This is a comment in JSX */}
  <p>Hello</p>
</div>
```

وافتكر : **JavaScript.** بس جوه الجافاسكريبت .أي حاجة بين `{ }`بتتنفذ كـ **HTML** شبه الـ **JSX** بالمصري **:الـEG
بنكتب **`className`** مش **`class`**. *In English: JSX is like HTML inside JavaScript. Anything inside `{ }`**

runs as JavaScript. Remember: write `className`, not `class`.*

🧪 Try it: Make a component that shows your name, your city, and your age.

0.07 5. 📦 Components

A component is a JavaScript function that returns JSX.

The name must start with a Capital letter.

0.07.01 Example 1: Simple component

```
function Hello() {
  return <h1>Hello!</h1>;
}

export default Hello;
```

0.07.02 Example 2: Use a component

```
import Hello from "./Hello";

function App() {
  return (
    <div>
      <Hello />
      <Hello />
      <Hello />
    </div>
  );
}
```

This shows "Hello!" three times. That is the power of components: write once, use many times.

0.07.03 Example 3: Arrow function style

```
const Footer = () => {
  return <footer>© 2026 My Website</footer>;
};
```

0.07.04 Example 4: Components inside components

```
function Header() {
  return <header><h1>My Shop</h1></header>;
}

function Footer() {
  return <footer><p>Thank you!</p></footer>;
}

function App() {
  return (
    <>
```

```
      <Header />
      <main><p>Welcome to the shop.</p></main>
      <Footer />
    </>
  );
}
```

0.07.05 Export types

Type	Export	Import
Default	`export default Card;`	`import Card from "./Card";`
Named	`export function Card() {}`	`import { Card } from "./Card";`

Table 5

⚠ **Mistake:** `function hello() {}` with a small `h`. React will think it is an HTML tag. Always use a **Capital letter**: `Hello`.

الفايدة إنك **(Capital).** لازم الاسم يبدأ بحرف كبير **JSX.** بترجّع **(function)** بالمصري **:الكومبوننت هو دالة** EG
بتكتبه مرة واحدة وتستخدمه في أماكن كتير. ***In English: A component is a function that returns JSX.***
The name must start with a capital letter. You write it once and use it many times.

0.08 6. 🎁 Props

Props (short for "properties") are the data you send to a component.

Think of props as the inputs of a function.

0.08.01 Example 1: One prop

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

function App() {
  return (
    <div>
      <Greeting name="Sara" />
      <Greeting name="Omar" />
    </div>
  );
}
```

0.08.02 Example 2: Destructuring (cleaner)

```
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}
```

0.08.03 Example 3: Many props

```
function CarCard({ brand, model, price, year }) {
  return (
    <div className="card">
      <h2>{brand} {model}</h2>
      <p>Year: {year}</p>
      <p>Price: ${price} per day</p>
    </div>
  );
}

function App() {
  return (
    <>
      <CarCard brand="Toyota" model="Corolla" price={40} year={2024} />
      <CarCard brand="Kia" model="Sportage" price={65} year={2025} />
    </>
  );
}
```

0.08.04 Strings vs other types

```
<CarCard brand="Toyota" />        {/* string → use quotes */}
<CarCard price={40} />            {/* number → use { } */}
<CarCard available={true} />      {/* boolean → use { } */}
<CarCard features={["AC", "GPS"]} />  {/* array → use { } */}
<CarCard owner={{ name: "Ali" }} />   {/* object → use {{ }} */}
```

0.08.05 Example 4: Default values

```
function Button({ text = "Click me", color = "gold" }) {
  return <button style={{ background: color }}>{text}</button>;
}

<Button />                        {/* "Click me", gold */}
<Button text="Save" />            {/* "Save", gold */}
<Button text="Delete" color="red" />
```

0.08.06 Example 5: Send a function as a prop

```
function Child({ onSayHi }) {
  return <button onClick={onSayHi}>Say Hi</button>;
}

function Parent() {
  function handleHi() {
    alert("Hi from parent!");
  }
  return <Child onSayHi={handleHi} />;
}
```

0.08.07 Example 6: The spread operator

```
const carData = { brand: "BMW", model: "X5", price: 120 };

<CarCard {...carData} year={2025} />
```

0.08.08 🟤 Golden Rule: Props are READ-ONLY

✕ Bad

```
function Bad({ name }) {
  name = "Changed";   // Do NOT do this!
  return <p>{name}</p>;
}
```

بتاعة الدالة. **parameters** زي الـ **(child)**، بيبعتها للابن **(parent)** هي البيانات اللي الأب بالمصري **props** :الـ** EG
الابن **يقرأها بس **ولا يغيّرها أبداً. ***In English: Props are data the parent sends to the child, like function parameters. The child can only read them. Never change them.***

🧪 Try it: Make a `ProductCard` component with `name`, `price`, and `image` props.

0.09 7. 🐵 Children

`children` **is a special prop. It is the content between the opening and closing tags.**

0.09.01 Example 1: A Box component

```
function Box({ children }) {
  return <div className="box">{children}</div>;
}

function App() {
  return (
    <Box>
      <h2>Title</h2>
      <p>Any content can go here.</p>
    </Box>
  );
}
```

0.09.02 Example 2: A Card with a title

```
function Card({ title, children }) {
  return (
    <section className="card">
      <h3>{title}</h3>
      <div>{children}</div>
    </section>
  );
}

<Card title="Rules">
  <ul>
    <li>Be kind</li>
    <li>Be on time</li>
  </ul>
</Card>
```

0.09.03 Example 3: A layout component

```
function Layout({ children }) {
  return (
    <>
      <nav>Menu</nav>
      <main>{children}</main>
      <footer>Footer</footer>
    </>
  );
}
```

هي أي حاجة بتكتبها **بين **فتحة وقفلة الكومبوننت .كده تقدر تعمل صندوق `children` بالمصري **:الـ** EG
وتحط جواه أي محتوى Layout أو (Box). *In English: `children` is anything you write between the opening and closing tags. So you can build a box or layout and put any content inside.*

0.10 8. 🧠 State with useState

State is the memory of a component.

When state changes, React updates the screen again.

	Props	State
Who owns it?	Parent	The component itself
Can it change?	❌ No	✅ Yes
Example	A name from the parent	A counter number

Table 6

0.10.01 Basic syntax

```
import { useState } from "react";

const [value, setValue] = useState(initialValue);
```

- `value` → the current value
- `setValue` → the function that changes it
- `initialValue` → the first value

0.10.02 Example 1: Counter

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(count - 1)}>-1</button>
      <button onClick={() => setCount(0)}>Reset</button>
```

```
    </div>
  );
}
```

0.10.03 Example 2: Text state

```
function NameBox() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Your name"
      />
      <p>Hello, {name || "stranger"}!</p>
    </div>
  );
}
```

0.10.04 Example 3: Boolean state (show / hide)

```
function Toggle() {
  const [show, setShow] = useState(false);

  return (
    <div>
      <button onClick={() => setShow(!show)}>
        {show ? "Hide" : "Show"}
      </button>
      {show && <p>Now you can see me! 👀</p>}
    </div>
  );
}
```

0.10.05 Example 4: Object state

```
function Profile() {
  const [user, setUser] = useState({ name: "Ali", age: 20 });

  function birthday() {
    // Make a NEW object. Do not change the old one.
    setUser({ ...user, age: user.age + 1 });
  }

  return (
    <div>
      <p>{user.name} is {user.age}</p>
      <button onClick={birthday}>Birthday 🎂</button>
    </div>
  );
}
```

0.10.06 Example 5: Array state

```
function Fruits() {
  const [fruits, setFruits] = useState(["Apple", "Banana"]);

  function addFruit() {
```

```
    setFruits([...fruits, "Orange"]);               // add
  }
  function removeFirst() {
    setFruits(fruits.slice(1));                     // remove first
  }
  function removeByName(name) {
    setFruits(fruits.filter((f) => f !== name));    // remove by name
  }
  function makeCapital() {
    setFruits(fruits.map((f) => f.toUpperCase()));  // change all
  }

  return (
    <div>
      <ul>{fruits.map((f) => <li key={f}>{f}</li>)}</ul>
      <button onClick={addFruit}>Add</button>
      <button onClick={removeFirst}>Remove first</button>
      <button onClick={makeCapital}>CAPITAL</button>
    </div>
  );
}
```

0.10.07 Example 6: Update using the previous value

When the new value depends on the old value, use a function:

```
function Triple() {
  const [count, setCount] = useState(0);

  function addThree() {
    setCount((prev) => prev + 1);
    setCount((prev) => prev + 1);
    setCount((prev) => prev + 1);
  }

  return <button onClick={addThree}>Count: {count}</button>;
}
```

This adds 3 each click. ✅

If you write `setCount(count + 1)` three times, it adds only 1. ❌

0.10.08 🟤 Never change state directly

❌ **Bad**

```
user.age = 21;          // direct change
fruits.push("Orange");  // direct change
```

✅ **Good**

```
setUser({ ...user, age: 21 });
setFruits([...fruits, "Orange"]);
```

`setSomething`، React بتعيد رسم الشاشة. هو ذاكرة الكومبوننت. لما بتغيّره بـ **state** بالمصري **:**الـ EG
(new copy) دايماً. ***In English: State is the component's memory. When you change it with `setSomething`, React draws the screen again. Never change** مباشرة — اعمل نسخة جديدة **state** ماتعدّلش الـ

state directly — always make a new copy.*

🧪 Try it: Make a "Like" button. Show a ♡ when liked and ♡ when not liked.

0.11 9. 👆 Events

Events are things the user does: click, type, submit, hover.

0.11.01 Common events

Event	When it runs
onClick	User clicks
onChange	Input value changes
onSubmit	Form is sent
onMouseEnter	Mouse goes over
onKeyDown	User presses a key
onFocus / onBlur	Input gets / loses focus

Table 7

0.11.02 Example 1: Click

```
function ClickMe() {
  function handleClick() {
    alert("You clicked!");
  }
  return <button onClick={handleClick}>Click me</button>;
}
```

0.11.03 Example 2: Inline arrow function

```
<button onClick={() => alert("Hi!")}>Say Hi</button>
```

0.11.04 Example 3: Send an argument

```
function Greet() {
  function sayHello(name) {
    alert("Hello " + name);
  }

  return (
    <>
      <button onClick={() => sayHello("Ali")}>Ali</button>
      <button onClick={() => sayHello("Mona")}>Mona</button>
    </>
  );
}
```

0.11.05 Example 4: The event object

```
function Typing() {
  const [text, setText] = useState("");
```

```
  return (
    <input
      value={text}
      onChange={(e) => setText(e.target.value)}
    />
  );
}
```

0.11.06 Example 5: Key press

```
function SearchBox() {
  function handleKey(e) {
    if (e.key === "Enter") {
      alert("Searching for: " + e.target.value);
    }
  }
  return <input onKeyDown={handleKey} placeholder="Press Enter" />;
}
```

0.11.07 Example 6: Stop the page refresh

```
function MyForm() {
  function handleSubmit(e) {
    e.preventDefault();   // stops the page from reloading
    alert("Form sent!");
  }
  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Send</button>
    </form>
  );
}
```

0.11.08 ⚠ Big mistake: calling the function

✕ Bad — runs immediately when the page loads

```
<button onClick={handleClick()}>Click</button>
```

☑ Good — runs only when the user clicks

```
<button onClick={handleClick}>Click</button>
<button onClick={() => handleClick(5)}>Click</button>
```

لأن كده الدالة هتشتغل فوراً مع تحميل `onClick`، بقوسين جوه `handleClick()` بالمصري **:ماتكتبش** EG
من غير أقواس، أو استخدم `() => handleClick(5)`. *In English: Do not write `handleClick()` with brackets inside `onClick`. It will run right away when the page loads. Write `handleClick` without brackets, or use `() => handleClick(5)`.*
الصفحة. اكتب `handleClick`

0.12 10. ⤮ Conditional Rendering

Show different things in different situations.

0.12.01 Way 1: `if` statement

```
function Status({ isOnline }) {
  if (isOnline) {
    return <p>🟢 Online</p>;
  }
  return <p>⚪ Offline</p>;
}
```

0.12.02 Way 2: Ternary `? :`

```
function Status({ isOnline }) {
  return <p>{isOnline ? "🟢 Online" : "⚪ Offline"}</p>;
}
```

0.12.03 Way 3: `&&` (show or nothing)

```
function Inbox({ count }) {
  return (
    <div>
      <h2>Inbox</h2>
      {count > 0 && <p>You have {count} new messages.</p>}
    </div>
  );
}
```

> ⚠ **Careful with numbers:** `{count && <p>...</p>}` shows **0** on the screen when count is 0. Use `count > 0 &&` instead.

0.12.04 Way 4: Variable

```
function Message({ user }) {
  let content;
  if (user) {
    content = <p>Welcome back, {user.name}!</p>;
  } else {
    content = <p>Please log in.</p>;
  }
  return <div>{content}</div>;
}
```

0.12.05 Example: Loading / Error / Data

```
function Page({ loading, error, data }) {
  if (loading) return <p>⏳ Loading...</p>;
  if (error) return <p>❌ Something went wrong.</p>;
  return <p>✅ Data: {data}</p>;
}
```

0.12.06 Example: Login button

```
function LoginButton() {
  const [loggedIn, setLoggedIn] = useState(false);

  return (
    <button onClick={() => setLoggedIn(!loggedIn)}>
```

```
      {loggedIn ? "Log out" : "Log in"}
    </button>
  );
}
```

EG **الـ:** بالمصري **Conditional Rendering** بنستخدم. شرط على بناءً معينة حاجة تعرض معناها **`if`** أو `:` ?`أو **`&&`. *In English: Conditional rendering means showing something based on a condition. We use `if`, `? :`, or `&&`.***

0.13 11. 📋 Lists and Keys

Use `.map()` to show a list of items.

0.13.01 Example 1: Simple list

```
function Colors() {
  const colors = ["Red", "Green", "Blue"];

  return (
    <ul>
      {colors.map((color) => (
        <li key={color}>{color}</li>
      ))}
    </ul>
  );
}
```

0.13.02 Example 2: List of objects

```
function CarList() {
  const cars = [
    { id: 1, brand: "Toyota", price: 40 },
    { id: 2, brand: "Kia", price: 55 },
    { id: 3, brand: "BMW", price: 120 },
  ];

  return (
    <div>
      {cars.map((car) => (
        <div key={car.id} className="card">
          <h3>{car.brand}</h3>
          <p>${car.price} per day</p>
        </div>
      ))}
    </div>
  );
}
```

0.13.03 Example 3: Use your own component

```
{cars.map((car) => (
  <CarCard key={car.id} brand={car.brand} price={car.price} />
))}
```

0.13.04 Example 4: Filter then map

```
const cheapCars = cars.filter((car) => car.price < 100);

return (
  <ul>
    {cheapCars.map((car) => (
      <li key={car.id}>{car.brand}</li>
    ))}
  </ul>
);
```

0.13.05 Example 5: Empty list message

```
{cars.length === 0 ? (
  <p>No cars found.</p>
) : (
  cars.map((car) => <p key={car.id}>{car.brand}</p>)
)}
```

0.13.06 🔑 What is a key?

A key helps React know which item is which when the list changes.

Key choice	
A unique ID (`item.id`)	☑ Best
A unique value (`item.email`)	☺ OK
Array index (`index`) if the list can change order	✕ Avoid

Table 8

```
// ✕ Avoid this when items can be added, removed, or moved
{items.map((item, index) => <li key={index}>{item}</li>)}

// ☑ Use this
{items.map((item) => <li key={item.id}>{item.name}</li>)}
```

تعرف مين اللي اتغيّر **React** لكل عنصر في اللستة. بيساعد **(ID card)** زي رقم البطاقة **`key`** بالمصري **:الـ** EG
فريد، مش رقم الـ **`id`** أو اتحذف. الأفضل تستخدم **index. *In English: A key is like an ID card for each item in the list. It helps React know which item changed or was removed. It is best to use a unique `id`, not the index number.***

0.14 12. 📝 Forms

In React, we usually use controlled inputs:

the input value lives in state.

0.14.01 Example 1: One input

```
function EmailForm() {
  const [email, setEmail] = useState("");
```

```
  function handleSubmit(e) {
    e.preventDefault();
    alert("Email: " + email);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="you@example.com"
      />
      <button type="submit">Send</button>
    </form>
  );
}
```

0.14.02 Example 2: Many inputs, one state object

```
function SignUp() {
  const [form, setForm] = useState({ name: "", email: "", password: "" });

  function handleChange(e) {
    const { name, value } = e.target;
    setForm({ ...form, [name]: value });
  }

  function handleSubmit(e) {
    e.preventDefault();
    console.log(form);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} placeholder="Name" />
      <input name="email" value={form.email} onChange={handleChange} placeholder="Email" />
      <input name="password" type="password" value={form.password} onChange={handleChange}
placeholder="Password" />
      <button>Create account</button>
    </form>
  );
}
```

> 💡 **`[name]: value` is a **computed key**. The `name` attribute of the input tells React which field to update.**

0.14.03 Example 3: Checkbox

```
function Terms() {
  const [agree, setAgree] = useState(false);

  return (
    <label>
      <input
```

```
        type="checkbox"
        checked={agree}
        onChange={(e) => setAgree(e.target.checked)}
      />
      I agree to the terms
    </label>
  );
}
```

0.14.04 Example 4: Select (dropdown)

```
function CarType() {
  const [type, setType] = useState("sedan");

  return (
    <select value={type} onChange={(e) => setType(e.target.value)}>
      <option value="sedan">Sedan</option>
      <option value="suv">SUV</option>
      <option value="van">Van</option>
    </select>
  );
}
```

0.14.05 Example 5: Radio buttons

```
function Plan() {
  const [plan, setPlan] = useState("basic");

  return (
    <div>
      {["basic", "pro", "business"].map((p) => (
        <label key={p}>
          <input
            type="radio"
            name="plan"
            value={p}
            checked={plan === p}
            onChange={(e) => setPlan(e.target.value)}
          />
          {p}
        </label>
      ))}
    </div>
  );
}
```

0.14.06 Example 6: Simple validation

```
function Login() {
  const [email, setEmail] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    if (!email.includes("@")) {
      setError("Please write a correct email.");
      return;
    }
    setError("");
    alert("Welcome!");
```

```
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={(e) => setEmail(e.target.value)} />
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button>Login</button>
    </form>
  );
}
```

EG ** الـ قيمة في الفورم، بنخلّي :**بالمصري **input** الـ في محفوظة **state.** يكتب المستخدم كل ما، `onChange` بتحدّث الـ **state.** ده اسمه ***Controlled Input**. *In English: In a form, we keep the input value in state. When the user types, `onChange` updates the state. This is called a controlled input.***

0.15 13. ⏱ useEffect

`useEffect` **runs code after the screen is drawn.**

We use it for "side effects": fetching data, timers, changing the page title, and more.

0.15.01 Syntax

```
import { useEffect } from "react";

useEffect(() => {
  // code to run
  return () => {
    // cleanup (optional)
  };
}, [dependencies]);
```

0.15.02 The dependency array

Array	When the effect runs
No array	After every render ⚠
`[]` (empty)	Once, when the component first appears
`[a, b]`	First time, and when `a` or `b` changes

Table 9

0.15.03 Example 1: Run once

```
function Welcome() {
  useEffect(() => {
    console.log("Component is on the screen!");
  }, []);

  return <h1>Welcome</h1>;
}
```

0.15.04 Example 2: Change the page title

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Clicked ${count} times`;
  }, [count]);

  return <button onClick={() => setCount(count + 1)}>Click</button>;
}
```

0.15.05 Example 3: Timer with cleanup

```
function Clock() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setSeconds((s) => s + 1);
    }, 1000);

    return () => clearInterval(id);   // cleanup: stop the timer
  }, []);

  return <p>⏱ {seconds} seconds</p>;
}
```

0.15.06 Example 4: Listen to the window

```
function WindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    function handleResize() {
      setWidth(window.innerWidth);
    }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return <p>Width: {width}px</p>;
}
```

0.15.07 Example 5: Save to localStorage

```
function Notes() {
  const [note, setNote] = useState(() => localStorage.getItem("note") || "");

  useEffect(() => {
    localStorage.setItem("note", note);
  }, [note]);

  return <textarea value={note} onChange={(e) => setNote(e.target.value)} />;
}
```

0.15.08 ⚠ Infinite loop

✕ Bad

```
useEffect(() => {
  setCount(count + 1);   // changes state → render → effect → ...
});                      // no dependency array
```

✅ Good

```
useEffect(() => {
  setCount(10);
}, []);                  // runs one time
```

0.15.09 💡 You may not need an effect

If you can calculate a value from props or state, do not use an effect.

```
// ❌ Not needed
const [fullName, setFullName] = useState("");
useEffect(() => setFullName(first + " " + last), [first, last]);

// ✅ Better
const fullName = first + " " + last;
```

بيشتغل **بعد **ما الشاشة تتعرض .بنستخدمه لجلب البيانات، التايمر، أو تغيير `useEffect`:** EG **بالمصري جواه بيتنفذ لما الكومبوننت يتشال `return` عنوان الصفحة .المصفوفة `[]` معناها "شغّله مرة واحدة بس ."والـ **(cleanup). *In English: `useEffect` runs after the screen is shown. We use it for fetching data, timers, or changing the page title. `[]` means "run only once". The `return` inside runs when the component is removed (cleanup).***

0.16 14. 🌐 Fetching Data

Get data from an API (a server).

0.16.01 Example 1: Basic fetch

```
import { useState, useEffect } from "react";

function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadUsers() {
      try {
        const res = await fetch("https://jsonplaceholder.typicode.com/users");
        if (!res.ok) throw new Error("Server error");
        const data = await res.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
```

```
    }
    loadUsers();
  }, []);

  if (loading) return <p>⏳ Loading...</p>;
  if (error) return <p>❌ {error}</p>;

  return (
    <ul>
      {users.map((u) => (
        <li key={u.id}>{u.name} — {u.email}</li>
      ))}
    </ul>
  );
}
```

0.16.02 The 3 states of every request

State	What to show
`loading`	A spinner or "Loading..."
`error`	A clear error message
`data`	The real content

Table 10

0.16.03 Example 2: Fetch when an ID changes

```
function UserDetails({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    let ignore = false;   // stops old requests from updating state

    fetch(`https://jsonplaceholder.typicode.com/users/${userId}`)
      .then((res) => res.json())
      .then((data) => {
        if (!ignore) setUser(data);
      });

    return () => {
      ignore = true;
    };
  }, [userId]);

  if (!user) return <p>Loading...</p>;
  return <h2>{user.name}</h2>;
}
```

0.16.04 Example 3: Send data with POST

```
async function createPost() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title: "Hello", body: "My first post", userId: 1 }),
  });
  const data = await res.json();
  console.log(data);
}
```

💡 **For big apps:** Many teams use **TanStack Query** (React Query). It gives you caching, retry, and loading states with less code.

EG **لما تجيب داتا من الـ API بالمصري:** دايماً جهّز 3 حالات: **loading** (بيحمّل)، **error** (حصل خطأ)، و **data** (البيانات جت). وده بيخلّي تجربة المستخدم أحسن. *In English: When you get data from an API, always prepare 3 states: loading, error, and data. This makes the user experience better.*

0.17 15. 📌 useRef

`useRef` **gives you a box that keeps a value.**

Changing it does not re-draw the screen.

0.17.01 Use 1: Focus an input

```
import { useRef } from "react";

function FocusInput() {
  const inputRef = useRef(null);

  return (
    <>
      <input ref={inputRef} placeholder="Type here" />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
    </>
  );
}
```

0.17.02 Use 2: Keep a value between renders

```
function RenderCounter() {
  const renders = useRef(0);
  renders.current += 1;
  return <p>Rendered {renders.current} times</p>;
}
```

0.17.03 Use 3: Save a timer ID

```
function Stopwatch() {
  const [time, setTime] = useState(0);
  const timerId = useRef(null);

  function start() {
    timerId.current = setInterval(() => setTime((t) => t + 1), 1000);
  }
  function stop() {
    clearInterval(timerId.current);
  }
```

```
  return (
    <>
      <p>{time}s</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </>
  );
}
```

0.17.04 `useState` vs `useRef`

	useState	useRef
Change → re-draw?	✅ Yes	❌ No
Use for	Things the user sees	Hidden values, DOM elements

Table 11

EG **بالمصري:** **`useRef`** زي علبة بتحفظ فيها حاجة. لو غيّرت اللي جواها، الشاشة **مش **بتتحدث. بنستخدمه أو نحفظ قيمة مخفية (زي ما نعمل **focus** على **input**) عشان نوصل لعنصر **HTML**. ***In English: `useRef` is like a box where you keep something. If you change what is inside, the screen does not update. We use it to reach an HTML element (like focusing an input) or to keep a hidden value.***

0.18 16. 🌍 useContext

Context lets you share data with many components without sending props through every level.

Problem: "Prop drilling"

```
App (has theme)
 └─ Page         ← must pass theme
     └─ Section  ← must pass theme
         └─ Button  ← finally uses theme
```

Solution: Context — Button reads `theme` directly.

0.18.01 Step 1: Create the context

```
// ThemeContext.jsx
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("dark");

  function toggleTheme() {
    setTheme((t) => (t === "dark" ? "light" : "dark"));
  }

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
```

```
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  return useContext(ThemeContext);
}
```

0.18.02 Step 2: Wrap your app

```
// main.jsx
import { ThemeProvider } from "./ThemeContext";

createRoot(document.getElementById("root")).render(
  <ThemeProvider>
    <App />
  </ThemeProvider>
);
```

0.18.03 Step 3: Use it anywhere

```
import { useTheme } from "./ThemeContext";

function ThemeButton() {
  const { theme, toggleTheme } = useTheme();

  return (
    <button onClick={toggleTheme}>
      Now: {theme} (click to change)
    </button>
  );
}
```

0.18.04 Example: User context

```
const UserContext = createContext(null);

function App() {
  const user = { name: "Mona", role: "admin" };
  return (
    <UserContext.Provider value={user}>
      <Dashboard />
    </UserContext.Provider>
  );
}

function Dashboard() {
  const user = useContext(UserContext);
  return <h1>Hi {user.name} ({user.role})</h1>;
}
```

بيخليك تشارك بيانات)زي الثيم أو المستخدم الحالي (مع كل الكومبوننتس من غير ما **Context** بالمصري **:الـ** EG تبعتها **props** من أب لابن لحفيد ...ده بيحل مشكلة **Prop Drilling**. *In English: Context lets you share data (like theme or current user) with all components without passing props from parent to child to grandchild. It solves the "prop drilling" problem.*

0.19 17. 🔄 useReducer

`useReducer` **is like** `useState`**, but better for complex state with many actions.**

0.19.01 Syntax

```
const [state, dispatch] = useReducer(reducer, initialState);
```

- **`reducer` → a function: `(state, action) => newState`**
- **`dispatch` → send an action**
- **`action` → a small object like `{ type: "add" }`**

0.19.02 Example 1: Counter

```
import { useReducer } from "react";

function reducer(state, action) {
  switch (action.type) {
    case "increase":
      return { count: state.count + 1 };
    case "decrease":
      return { count: state.count - 1 };
    case "reset":
      return { count: 0 };
    default:
      return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <div>
      <h2>{state.count}</h2>
      <button onClick={() => dispatch({ type: "increase" })}>+</button>
      <button onClick={() => dispatch({ type: "decrease" })}>-</button>
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </div>
  );
}
```

0.19.03 Example 2: Shopping cart

```
function cartReducer(state, action) {
  switch (action.type) {
    case "add":
      return [...state, action.item];
    case "remove":
      return state.filter((item) => item.id !== action.id);
    case "clear":
      return [];
    default:
      return state;
  }
```

```
}

function Cart() {
  const [cart, dispatch] = useReducer(cartReducer, []);

  return (
    <div>
      <button onClick={() => dispatch({ type: "add", item: { id: Date.now(), name: "Car" } })}>
        Add
      </button>
      <button onClick={() => dispatch({ type: "clear" })}>Clear</button>
      <ul>
        {cart.map((item) => (
          <li key={item.id}>
            {item.name}
            <button onClick={() => dispatch({ type: "remove", id: item.id })}>✕</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

0.19.04 When to use which?

Situation	Use
One simple value	useState
A few separate values	useState
Many related values and actions	useReducer
Next state depends on complex rules	useReducer

Table 12

يبقى معقّد وفيه حاجات كتير بتحصل عليه)إضافة، حذف، **state** مناسب لما الـ **`useReducer`** :**بالمصري** EG
"تعديل .(بدل ما تكتب دوال كتير، بتبعت "أكشن **(action) والـ **reducer** هو اللي يقرر الـ **state** الجديد. ***In English: `useReducer` is good when state is complex and many things can happen to it (add, remove, edit). Instead of many functions, you send an action and the reducer decides the new state.***

0.20 18. 🏎️ Performance: memo, useMemo, useCallback

🟤 ****Important:** Do **not** use these everywhere. React is fast already. Use them only when you **see** a slow screen.**

0.20.01 `React.memo` — skip re-drawing if props are the same

```
import { memo } from "react";
```

```
const CarCard = memo(function CarCard({ brand, price }) {
  console.log("Drawing", brand);
  return <div>{brand} — ${price}</div>;
});
```

0.20.02 `useMemo` — remember a calculated value

```
import { useMemo, useState } from "react";

function ExpensiveList({ numbers }) {
  const [dark, setDark] = useState(false);

  // This runs again only when `numbers` changes
  const total = useMemo(() => {
    console.log("Calculating...");
    return numbers.reduce((sum, n) => sum + n, 0);
  }, [numbers]);

  return (
    <div style={{ background: dark ? "#222" : "#fff" }}>
      <p>Total: {total}</p>
      <button onClick={() => setDark(!dark)}>Change color</button>
    </div>
  );
}
```

0.20.03 `useCallback` — remember a function

```
import { useCallback, useState, memo } from "react";

const Button = memo(function Button({ onClick, label }) {
  console.log("Button drawn:", label);
  return <button onClick={onClick}>{label}</button>;
});

function Parent() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  const increase = useCallback(() => {
    setCount((c) => c + 1);
  }, []);

  return (
    <>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <p>{count}</p>
      <Button onClick={increase} label="+1" />
    </>
  );
}
```

Now typing in the input does not re-draw `Button`. ☑

0.20.04 Quick summary

Tool	Remembers	Use when
`memo`	A component	Props rarely change
`useMemo`	A value	Heavy calculation

Tool	Remembers	Use when
`useCallback`	A function	Sending a function to a `memo` child

Table 13

0.20.05 Other easy performance tips

- ✅ **Use a correct `key` in lists.**
- ✅ **Split big components into smaller ones.**
- ✅ **Use **lazy loading** for big pages:**

```
import { lazy, Suspense } from "react";

const AdminPage = lazy(() => import("./AdminPage"));

function App() {
  return (
    <Suspense fallback={<p>Loading page...</p>}>
      <AdminPage />
    </Suspense>
  );
}
```

ماتعملش شغل زيادة . **React** أدوات "تذكّر" عشان **`useCallback`** و **`useMemo`** و **`memo`**:** بالمصري** EG بس ماتستخدمهاش في كل حاجة — استخدمها بس لما تلاقي الصفحة بطيئة فعلاً. ***In English: `memo`, `useMemo`, and `useCallback` are "remember" tools so React does not do extra work. But do not use them everywhere — use them only when the page is really slow.***

0.21 19. □ Custom Hooks

A custom hook is your own function that uses other hooks.

The name must start with `use`.

0.21.01 Example 1: `useToggle`

```
import { useState } from "react";

function useToggle(initial = false) {
  const [value, setValue] = useState(initial);
  const toggle = () => setValue((v) => !v);
  return [value, toggle];
}

// Use it
function Menu() {
  const [open, toggleOpen] = useToggle();
  return (
    <>
      <button onClick={toggleOpen}>Menu</button>
      {open && <nav>Home | About | Contact</nav>}
```

```
    </>
  );
}
```

0.21.02 Example 2: `useFetch`

```
import { useState, useEffect } from "react";

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let ignore = false;
    setLoading(true);

    fetch(url)
      .then((res) => {
        if (!res.ok) throw new Error("Request failed");
        return res.json();
      })
      .then((json) => !ignore && setData(json))
      .catch((err) => !ignore && setError(err.message))
      .finally(() => !ignore && setLoading(false));

    return () => {
      ignore = true;
    };
  }, [url]);

  return { data, loading, error };
}

// Use it
function Posts() {
  const { data, loading, error } = useFetch("https://jsonplaceholder.typicode.com/posts?_limit=5");

  if (loading) return <p>Loading...</p>;
  if (error) return <p>{error}</p>;
  return <ul>{data.map((p) => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

0.21.03 Example 3: `useLocalStorage`

```
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Use it — same as useState, but it remembers after refresh!
const [name, setName] = useLocalStorage("name", "");
```

0.21.04 Example 4: `useDebounce` (good for search boxes)

```
function useDebounce(value, delay = 500) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);

  return debounced;
}

function Search() {
  const [text, setText] = useState("");
  const search = useDebounce(text, 500);

  useEffect(() => {
    if (search) console.log("Search API for:", search);
  }, [search]);

  return <input value={text} onChange={(e) => setText(e.target.value)} />;
}
```

EG **الـ:** بالمصري **Custom Hook** هو دالة بتكتبها إنت، اسمها لازم يبدأ بـ **`use`**، وبتجمّع جواها منطق **(logic)** بتكرره في أكتر من مكان. كده الكود بيبقى نضيف ومتنظّم. ***In English: A custom hook is a function you write. Its name must start with `use`. It holds logic you repeat in many places. This keeps your code clean and organized.***

0.22 20. ⚛ React Router

React Router changes pages without reloading the browser.

0.22.01 Install

```
npm install react-router-dom
```

0.22.02 Example 1: Basic routes

```
// main.jsx
import { BrowserRouter } from "react-router-dom";

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

```
// App.jsx
import { Routes, Route, Link } from "react-router-dom";

function Home() { return <h1>🏠 Home</h1>; }
```

```
function About() { return <h1>ℹ About</h1>; }
function NotFound() { return <h1>404 – Page not found</h1>; }

function App() {
  return (
    <>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </>
  );
}
```

0.22.03 Example 2: URL parameters

```
import { useParams } from "react-router-dom";

function CarDetails() {
  const { id } = useParams();
  return <h1>Car number: {id}</h1>;
}

<Route path="/cars/:id" element={<CarDetails />} />
// Visit /cars/7 → "Car number: 7"
```

0.22.04 Example 3: Go to another page with code

```
import { useNavigate } from "react-router-dom";

function Login() {
  const navigate = useNavigate();

  function handleLogin() {
    // ... check user
    navigate("/dashboard");
  }

  return <button onClick={handleLogin}>Login</button>;
}
```

0.22.05 Example 4: Active link style

```
import { NavLink } from "react-router-dom";

<NavLink
  to="/about"
  style={({ isActive }) => ({ color: isActive ? "gold" : "white" })}
>
  About
</NavLink>
```

0.22.06 Example 5: Protected page

```
import { Navigate } from "react-router-dom";

function ProtectedRoute({ isLoggedIn, children }) {
  if (!isLoggedIn) return <Navigate to="/login" replace />;
  return children;
}

<Route
  path="/dashboard"
  element={
    <ProtectedRoute isLoggedIn={user !== null}>
      <Dashboard />
    </ProtectedRoute>
  }
/>
```

0.22.07 Example 6: Nested routes

```
import { Outlet } from "react-router-dom";

function DashboardLayout() {
  return (
    <div>
      <aside>Sidebar</aside>
      <section><Outlet /></section>   {/* child page appears here */}
    </div>
  );
}

<Route path="/dashboard" element={<DashboardLayout />}>
  <Route index element={<Overview />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

> ⚠ **Use `<Link>`, not `<a href>`. A normal `<a>` reloads the whole page.**

> استخدم **reload.** بيخليك تتنقل بين الصفحات من غير ما المتصفح يعمل **React Router** بالمصري **:الـEG** **`<Link>`** بدل **`<a>`**، و **`useParams`** عشان تقرأ الـ **id** من اللينك. ***In English: React Router lets you move between pages without a browser reload. Use `<Link>` instead of `<a>`, and `useParams` to read the id from the link.***

0.23 21. 🎨 Styling

0.23.01 Way 1: Normal CSS file

```
// Button.jsx
import "./Button.css";
```

```
function Button() {
  return <button className="gold-btn">Click</button>;
}
```

```
/* Button.css */
.gold-btn {
  background: #d4af37;
  color: #111;
  border: none;
  padding: 10px 20px;
  border-radius: 8px;
  cursor: pointer;
}
.gold-btn:hover {
  background: #f1c40f;
}
```

0.23.02 Way 2: Inline style

```
<div style={{ background: "#111", color: "#d4af37", padding: 16 }}>
  Dark and gold ♥♡
</div>
```

0.23.03 Way 3: CSS Modules (no name conflicts)

```
/* Card.module.css */
.card {
  border: 1px solid #d4af37;
  border-radius: 12px;
  padding: 16px;
}
```

```
import styles from "./Card.module.css";

function Card() {
  return <div className={styles.card}>Hello</div>;
}
```

0.23.04 Way 4: Tailwind CSS (utility classes)

```
<button className="bg-yellow-500 text-black px-4 py-2 rounded-lg hover:bg-yellow-400">
  Click
</button>
```

0.23.05 Dynamic classes

```
function Tab({ active, children }) {
  return (
    <button className={active ? "tab active" : "tab"}>
      {children}
    </button>
  );
}

// Or with template string
<div className={`card ${selected ? "card-selected" : ""}`} />
```

0.23.06 Which one should I choose?

Way	Good for
Normal CSS	Small projects, learning
Inline style	Quick, dynamic values
CSS Modules	Team projects, no conflicts
Tailwind	Fast building, consistent design

Table 14

CSS عادي .ولما تكبر، جرّب **CSS** للمبتدئ :ابدأ بملف **styling.** عندك طرق كتير للـ **React** بالمصري **:**في**
EG
Modules أو **Tailwind. *In English: In React you have many ways to style. As a beginner, start with a normal CSS file. When you grow, try CSS Modules or Tailwind.***

0.24 22. ⬆ Lifting State Up

Two components need the same data? Move the state to their closest common parent.

0.24.01 Example: Two components share a temperature

```
function Input({ value, onChange }) {
  return (
    <input
      type="number"
      value={value}
      onChange={(e) => onChange(e.target.value)}
    />
  );
}

function Display({ celsius }) {
  const fahrenheit = (celsius * 9) / 5 + 32;
  return <p>{celsius}°C = {fahrenheit}°F</p>;
}

function Converter() {
  const [celsius, setCelsius] = useState(0);   // state lives in the parent

  return (
    <div>
      <Input value={celsius} onChange={setCelsius} />
      <Display celsius={celsius} />
    </div>
  );
}

        Converter  (has state)
        /        \
     Input      Display
  (changes it)  (shows it)
```

0.24.02 Example: Search + List

```
function SearchPage() {
  const [query, setQuery] = useState("");
  const items = ["Apple", "Banana", "Orange", "Mango"];
  const result = items.filter((i) =>
    i.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <ul>{result.map((i) => <li key={i}>{i}</li>)}</ul>
    </>
  );
}
```

🔘 **Data flows down (props). Events flow up (functions).**

EG **لو اتنين كومبوننتس محتاجين نفس الـ**:** بالمصري **state**، شيله وحطه في **الأب المشترك** بينهم. والأبناء بيبعتوا التغيير فوق عن طريق دوال، بيبعت القيمة تحت كـ **props**. ***In English: If two components need the same state, move it to their common parent. The parent sends the value down as props, and the children send changes up using functions.***

0.25 23. 📜 Rules of Hooks

Hooks are special. Follow these 2 rules:

0.25.01 Rule 1: Call hooks only at the **top level**

❌ Bad

```
function Bad({ show }) {
  if (show) {
    const [count, setCount] = useState(0);   // inside if
  }
  for (let i = 0; i < 3; i++) {
    useEffect(() => {});                      // inside loop
  }
}
```

✅ Good

```
function Good({ show }) {
  const [count, setCount] = useState(0);
  useEffect(() => {
    if (show) {
      // use the "if" inside the hook
    }
  }, [show]);
}
```

0.25.02 Rule 2: Call hooks only in **React functions**

- ✅ **Inside a component**
- ✅ **Inside a custom hook (name starts with `use`)**
- ❌ **Inside a normal JavaScript function**
- ❌ **Inside event handlers**

> EG **قاعدتين للـ:** بالمصري **Hooks: (1)** استدعيها في أول الكومبوننت، مش جوه **`if`** أو **`for`. (2)** استدعيها بس جوه كومبوننت أو **Custom Hook. *In English: Two rules for hooks: (1) Call them at the top of the component, not inside `if` or `for`. (2) Call them only inside a component or a custom hook.***

0.26 24. ⚠ Common Mistakes

#	❌ Mistake	✅ Fix
1	`function card() {}`	`function Card() {}` (Capital letter)
2	`class="box"`	`className="box"`
3	`onClick={fn()}`	`onClick={fn}` or `onClick={() => fn(1)}`
4	`state.push(x)`	`setState([...state, x])`
5	`user.name = "A"`	`setUser({ ...user, name: "A" })`
6	No `key` in a list	Add `key={item.id}`
7	`key={index}` in changing lists	Use a unique ID
8	Hook inside `if`	Move the hook to the top
9	`useEffect` with no `[]` and `setState` inside	Add the correct dependency array
10	`{count && <p/>}` when count can be 0	`{count > 0 && <p/>}`
11	Two parents in `return`	Wrap with `<>...</>`
12	Change props	Props are read-only
13	Forget `e.preventDefault()` in forms	Add it in `onSubmit`
14	Use `<a href>` for pages	Use `<Link to>`
15	Forget cleanup for timers	`return () => clearInterval(id)`

Table 15

0.26.01 Mistake 16: State does not change immediately

```
function Wrong() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    console.log(count);   // still the OLD value (0)!
  }
  return <button onClick={handleClick}>{count}</button>;
}
```

State updates after the function finishes. The new value is there in the next render.

> EG **بعد:** بالمصري **`setCount`**، القيمة الجديدة **مش **بتظهر فوراً في نفس الدالة. هتلاقيها في الرندر اللي بعده. ده من أكتر الحاجات اللي بتلخبط المبتدئين. ***In English: After `setCount`, the new value does not***

appear right away in the same function. You will see it in the next render. This confuses many beginners.*

0.27 25. 🏆 Mini Project: Todo App

Let's put everything together. This app can add, complete, delete, and filter tasks. It also saves to the browser.

0.27.01 `App.jsx`

```
import { useState, useEffect } from "react";
import "./App.css";

function App() {
  // 1) STATE
  const [todos, setTodos] = useState(() => {
    const saved = localStorage.getItem("todos");
    return saved ? JSON.parse(saved) : [];
  });
  const [text, setText] = useState("");
  const [filter, setFilter] = useState("all"); // all | active | done

  // 2) SAVE when todos change
  useEffect(() => {
    localStorage.setItem("todos", JSON.stringify(todos));
  }, [todos]);

  // 3) FUNCTIONS
  function addTodo(e) {
    e.preventDefault();
    const title = text.trim();
    if (!title) return;

    setTodos([...todos, { id: Date.now(), title, done: false }]);
    setText("");
  }

  function toggleTodo(id) {
    setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  }

  function deleteTodo(id) {
    setTodos(todos.filter((t) => t.id !== id));
  }

  function clearDone() {
    setTodos(todos.filter((t) => !t.done));
  }

  // 4) CALCULATED VALUES (no extra state needed)
  const visibleTodos = todos.filter((t) => {
    if (filter === "active") return !t.done;
```

```
    if (filter === "done") return t.done;
    return true;
  });
  const leftCount = todos.filter((t) => !t.done).length;

  // 5) UI
  return (
    <div className="app">
      <h1>✅ My Tasks</h1>

      <form onSubmit={addTodo} className="add-form">
        <input
          value={text}
          onChange={(e) => setText(e.target.value)}
          placeholder="What do you need to do?"
        />
        <button type="submit">Add</button>
      </form>

      <div className="filters">
        {["all", "active", "done"].map((f) => (
          <button
            key={f}
            className={filter === f ? "active" : ""}
            onClick={() => setFilter(f)}
          >
            {f}
          </button>
        ))}
      </div>

      {visibleTodos.length === 0 ? (
        <p className="empty">Nothing here 🎉</p>
      ) : (
        <ul>
          {visibleTodos.map((todo) => (
            <TodoItem
              key={todo.id}
              todo={todo}
              onToggle={() => toggleTodo(todo.id)}
              onDelete={() => deleteTodo(todo.id)}
            />
          ))}
        </ul>
      )}

      <footer>
        <span>{leftCount} task(s) left</span>
        <button onClick={clearDone}>Clear done</button>
      </footer>
    </div>
  );
}

function TodoItem({ todo, onToggle, onDelete }) {
  return (
    <li className={todo.done ? "done" : ""}>
      <label>
        <input type="checkbox" checked={todo.done} onChange={onToggle} />
```

```
        <span>{todo.title}</span>
      </label>
      <button onClick={onDelete} aria-label="Delete task">🗑️</button>
    </li>
  );
}

export default App;
```

0.27.02 `App.css` (dark and gold ❤️🤎)

```
body {
  margin: 0;
  background: #0d0d0d;
  color: #f5f5f5;
  font-family: system-ui, sans-serif;
}

.app {
  max-width: 480px;
  margin: 40px auto;
  padding: 24px;
  background: #1a1a1a;
  border: 1px solid #d4af37;
  border-radius: 16px;
}

h1 { color: #d4af37; text-align: center; }

.add-form { display: flex; gap: 8px; }
.add-form input {
  flex: 1;
  padding: 10px;
  border-radius: 8px;
  border: 1px solid #444;
  background: #111;
  color: #fff;
}

button {
  background: #d4af37;
  color: #111;
  border: none;
  padding: 8px 14px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 600;
}
button:hover { background: #f1c40f; }

.filters { display: flex; gap: 8px; margin: 16px 0; }
.filters button { background: #333; color: #ddd; }
.filters button.active { background: #d4af37; color: #111; }

ul { list-style: none; padding: 0; }
li {
  display: flex;
  justify-content: space-between;
  align-items: center;
```

```
  padding: 10px 0;
  border-bottom: 1px solid #333;
}
li.done span { text-decoration: line-through; opacity: 0.5; }

footer {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 16px;
  color: #aaa;
}
.empty { text-align: center; color: #888; }
```

0.27.03 What did we use?

Concept	Where in the project
useState	todos, text, filter
useEffect	Save to localStorage
Events	onChange, onSubmit, onClick
Forms	The add form
Lists + keys	visibleTodos.map(...)
Conditional rendering	Empty message
Props	TodoItem
Functions as props	onToggle, onDelete

Table 16

جرّب تضيف **useEffect.** و **state**، **events**، **forms**، **lists**: بالمصري **:المشروع ده بيجمّع كل اللي اتعلمناه** EG
In English: This project combines everything we learned: state, events, forms, lists, and useEffect. Try to add a new feature, like editing a task. المهمة **(edit)** ميزة جديدة، زي **تعديل**

0.28 26. Cheat Sheet

0.28.01 Hooks

Hook	One-line meaning	Example
useState	Memory for a component	const [n, setN] = useState(0)
useEffect	Run code after render	useEffect(() => {...}, [])
useRef	Box / DOM element	const r = useRef(null)
useContext	Read shared data	useContext(ThemeContext)
useReducer	Complex state	useReducer(reducer, init)
useMemo	Remember a value	useMemo(() => calc, [dep])
useCallback	Remember a function	useCallback(fn, [dep])

Table 17

0.28.02 Patterns

```
// Show / hide
{show && <Box />}
```

```
// Choose one
{ok ? <Yes /> : <No />}

// List
{items.map((i) => <Item key={i.id} {...i} />)}

// Update object
setUser({ ...user, name: "New" });

// Add to array
setList([...list, newItem]);

// Remove from array
setList(list.filter((i) => i.id !== id));

// Update one item in array
setList(list.map((i) => (i.id === id ? { ...i, done: true } : i)));

// Update using previous value
setCount((prev) => prev + 1);
```

0.28.03 File flow

```
index.html → main.jsx → App.jsx → Components → Hooks / Props / State
```

0.29 27. 📖 Glossary: English ↔ Egyptian Arabic

English	Egyptian Arabic	Easy meaning
Component	كومبوننت /قطعة	A small piece of UI
Props	بروبس /مدخلات	Data from parent to child
State	ستيت /الذاكرة	Data that can change
Hook	هوك	A special React function (`useXxx`)
Render	رندر /يرسم	Draw the UI on the screen
Re-render	يرسم تاني	Draw again after a change
Parent	الأب	The component that holds others
Child	الابن	A component inside another
Event	حدث)إيفنت(	Click, type, submit
Handler	دالة التعامل مع الحدث	Function that runs on an event
Key	مفتاح /رقم مميز	Unique ID for a list item
List	لستة /قائمة	Many items
Array	مصفوفة	A list in JavaScript
Object	أوبجكت /كائن	`{ name: "Ali" }`
Dependency	اعتمادية	A value an effect watches
Side effect	تأثير جانبي	Work outside the UI (API, timer)
Cleanup	تنضيف	Stop timers or listeners
Controlled input	إدخال متحكم فيه	Input value lives in state
Context	كونتكست /مشاركة بيانات	Share data to many components
Prop drilling	تمرير بروبس لمستويات كتير	Passing props through many levels
Route	مسار /صفحة	A page address
Fetch	يجيب بيانات	Get data from a server
API	واجهة برمجة	A server that gives data
Loading	بيحمّل	Waiting for data
Error	خطأ	Something went wrong
Fragment	فراجمنت	`<> </>` — a wrapper with no extra tag

English	Egyptian Arabic	Easy meaning
Virtual DOM	دوم افتراضي	React's fast copy of the page
Production	بيئة الإنتاج	The final live version

Table 18

0.30 28. 🏏 Practice Exercises

0.30.01 🟢 Easy

33. **Profile Card:** Make a `ProfileCard` component with props: `name`, `job`, `city`.
34. **Like Button:** A button that changes between ♡ and ♡.
35. **Counter:** Add buttons: `+1`, `-1`, `Reset`. The number must not go below 0.
36. **Show / Hide:** A button that shows and hides a paragraph.

0.30.02 🟡 Medium

37. **Car List:** Show 5 cars from an array. Add a search input to filter by brand.
38. **Contact Form:** Name, email, and message. Show an error if a field is empty.
39. **Tabs:** Three buttons (Home, About, Contact). Show different text for each.
40. **Shopping Cart:** Add items, remove items, and show the total price.

0.30.03 🔴 Hard

41. **Users from API:** Fetch users from `jsonplaceholder`. Show loading and error. Add a search box.
42. **Theme Switch:** Use Context to switch between dark and light mode in the whole app.
43. **Multi-page App:** Use React Router with Home, Cars list, and Car details (`/cars/:id`).
44. **Custom Hook:** Make `useLocalStorage` and use it in your Todo app.

> **section** بالمصري **:الشاطر اللي يكتب الكود بنفسه .ابدأ بالتمارين السهلة، وبعدين اتدرّج .لو علقت، ارجع للـ** EG المناسب في الملف ده. ***In English: The best way to write code yourself. Start with easy exercises, then go step by step. If you get stuck, go back to the right section in this file.***

0.31 29. 📬 Next Steps

After this guide, learn these in order:

45. **TypeScript** with React — fewer bugs.
46. **TanStack Query** — easy and powerful data fetching.
47. **Zustand** or **Redux Toolkit** — global state for bigger apps.
48. **React Hook Form** + **Zod** — better forms and validation.
49. **Next.js** — React framework (routing, server rendering, SEO).

50. **Testing** — Vitest and React Testing Library.
51. **Accessibility (a11y)** — make your app good for everyone.

0.31.01 Good habits ☑

- Keep components **small** (one job each).
- Use **clear names**: `UserCard`, not `Thing1`.
- Build a small project every week.
- Read error messages slowly. They often tell you the answer.
- Use the official docs: **react.dev**.

0.31.02 🏵 Happy coding! ❤️🩶

Practice every day. Small steps make big results.

> **In English: Start now, and do not be afraid of mistakes — mistakes are the fastest way to learn!** *يلا ابدأ، وماتخافش تغلط — الغلط هو أسرع طريقة للتعلّم* EG